Católica
Universidade Católica de Brasília

Documento de Visão

GRUPO 05

Versão

Revisões do Documento

Revisões são melhoramentos na estrutura do documento e também no seu conteúdo. O objetivo primário desta tabela é a fácil identificação da versão do documento. Toda modificação no documento deve constar nesta tabela.

Versão	Data	Autores	Observações

Auditorias do Documento

Auditorias são inspeções formais conduzidas pela equipe de Qualidade e Conformidade da Empresa, e tem por objetivo garantir a qualidade dos artefatos gerados durante o processo de desenvolvimento do mesmo. Toda modificação no documento deve constar nesta tabela.

Versão	Data	Autores	E-mail	Observações

1.1.1.1

Sumário

1. Introdução 4
1.1 Finalidade 4
1.2 Escopo 4
1.3 Definições, Acrônimos e Abreviações 4
1.4 Referências 4
1.4.1 Especificações Suplementares 4
2. Contextualização 5
2.1 Descrição do Problema 5
2.2 Sentença de Posição do Produto 5
3. Descrição dos *Stakeholders* e dos Usuários 5
3.1 Principais *Stakeholders* e Usuários 5
3.2 Necessidades Chave dos *Stakeholders* e dos Usuários 6
4. Visão Geral do Produto 7
4.1 Perspectiva do Produto 7
4.2 Estimativa de Custo Inicial 8
4.3 Suposições e dependências 8
4.4 Premissas e Dependências 8
5. Precedência e Prioridades 9
6. Requisitos Não-Funcionais do Produto 10
7. Restrições Técnicas 10
7.1 Padrões Aplicáveis 10
7.2 Restrições Aplicáveis 10
8. Manual de Arquitetura 10
8.1 Visão de Implementação 10
8.2 Mecanismo Arquiteturais 11
9. Dicionário de Dados 11

9.1 Finalidade 11

9.2 Nome da Tabela 11

Documento de Visão

2. Introdução

2.1 Finalidade

A finalidade deste Documento de Visão é coletar, analisar e definir as necessidades de alto nível e as funcionalidades do sistema ***UniCarona***. Ele tem como objetivo alinhar as expectativas entre todos os *stakeholders* (estudantes, passageiros, alunos motoristas, equipe de desenvolvimento e administração da plataforma), estabelecendo uma base comum de entendimento sobre o problema do desgaste no deslocamento diário e como o aplicativo propõe solucioná-lo.

Além disso, este documento serve para guiar as fases subsequentes do ciclo de vida do desenvolvimento de software, garantindo que o escopo permaneça focado na criação de uma rede de caronas solidárias que priorize a segurança institucional, a rastreabilidade e a ajuda mútua dentro da comunidade acadêmica.

2.2 Escopo

O projeto ***UniCarona*** contempla o desenvolvimento de uma plataforma (aplicativo) voltada para a intermediação de caronas solidárias exclusivas para a comunidade acadêmica. O sistema permitirá:

- O cadastro rigoroso de usuários (motoristas e passageiros), incluindo a validação de identidade (CPF e reconhecimento facial) e comprovação de vínculo universitário.
- A criação, unificação e visualização de rotas de deslocamento entre as cidades satélites e o campus.
- O cruzamento de dados (*matching*) para conectar passageiros a motoristas com trajetos compatíveis.
- O rastreamento das viagens e o registro de histórico detalhado para garantir a segurança institucional.
- Um sistema de avaliação bidirecional (motorista avalia passageiro e vice-versa) para ranqueamento e confiabilidade.

Foge ao escopo desta plataforma o processamento de transações financeiras com fins lucrativos (como em aplicativos de transporte comercial), a responsabilização por incidentes de trânsito físicos e o fornecimento dos veículos, sendo a plataforma restrita à facilitação do contato e monitoramento digital da carona solidária.

2.3 Definições, Acrônimos e Abreviações

Para a correta interpretação deste documento e do Glossário do Sistema, adotam-se as seguintes definições:

- **App:** Abreviação de Aplicativo (software desenvolvido para dispositivos móveis).
- **Carona Solidária:** Modalidade de compartilhamento de veículo sem fins lucrativos, cujo objetivo é apenas a cortesia ou a divisão igualitária de custos (como combustível).
- **Cidades Satélites:** Termo utilizado para designar as Regiões Administrativas mais distantes do centro, evidenciando as áreas de origem dos trajetos mais exaustivos dos alunos.
- **Matching:** Processo sistêmico/algoritmo que cruza os dados de rota e horários do motorista com a solicitação do passageiro para encontrar viagens compatíveis.
- **Stakeholders:** Partes interessadas que afetam ou são afetadas pelos resultados do projeto (passageiros, motoristas, administradores e desenvolvedores).
- **Facial ID / Reconhecimento Facial:** Tecnologia de segurança biométrica utilizada para validar a identidade do usuário no momento do cadastro ou antes do início das viagens.

2.4 Referências

As referências a seguir fornecem as diretrizes, restrições e detalhamentos complementares que guiam a construção e o escopo do UniCarona:

2.4.1 Especificações Suplementares

- Documento de Regra de Negócio (contendo as políticas de uso, normas de segurança e diretrizes de carona não-lucrativa).
- Documento de Requisitos (elicitar todos os requisitos funcionais e não funcionais do sistema, como o funcionamento do *matching* e da validação de segurança).
- Documento Manual do Usuário.
- Documento Dicionário de Dados.

2.4.2 Documentos externos

– **REF 01:** LIMA, Patrick; LIMA, Vitor. **RideUFF: Desenvolvimento de um aplicativo de carona solidária para a comunidade da Universidade Federal Fluminense**. Niterói: UFF, 2019. Disponível em: https://app.uff.br/riuff/handle/1/10982. Acesso em: 20 mar. 2026.

– REF 02: SOUZA, J. A. et al. **Carona Solidária: Uma Alternativa de Mobilidade e suas Vantagens no Ambiente Acadêmico**. Revista de Gestão e Tecnologia (IFES), 2018. Disponível em: https://www.researchgate.net/publication/326518113. Acesso em: 20 mar. 2026.

– REF 03: FERREIRA, L. M.; GOMES, R. S. **Adoção de serviços de carona solidária: análise dos motivadores e barreiras entre usuários urbanos**. Revista Gestão e Desenvolvimento (Feevale), v. 15, n. 2, 2018. Disponível em: https://periodicos.feevale.br/seer/index.php/revistagestaoedesenvolvimento/article/view/4150. Acesso em: 20 mar. 2026.

3. Contextualização

3.1 Descrição do Problema

Problemas	• Desgaste físico e mental de alunos que utilizam múltiplos modais de transporte público. • Incompatibilidade entre horários de ônibus/metrô e o término das aulas noturnas. • Dificuldade de deslocamento para alunos que moram longe. • Falta de uma rede de apoio organizada para deslocamento entre cidades satélites e o campus.
Pessoas Atingidas	• Estudantes que residem em cidades longes ou nas mesmas cidades satélites • Alunos condutores que possuem vagas ociosas em seus veículos.
Cujo impacto é	• Fadiga excessiva e redução do rendimento acadêmico devido ao tempo gasto no trânsito. • Atrasos frequentes em aulas de primeiro horário ou dificuldades para retornar após aulas noturnas. • Impacto financeiro relevante no orçamento mensal do estudante.
Uma solução bem-sucedida traria	• Fortalecimento do espírito de comunidade e ajuda mútua entre os estudantes. • Democratização do acesso à universidade para quem mora em regiões distantes. • Ajuda no aumento do rendimento acadêmico devido a uma rotina menos cansativa no trajeto de ida e volta no trânsito

3.2 Sentença de Posição do Produto

Para	estudantes da universidades que enfrentam trajetos exaustivos entre as cidades satélites e o campus.
O	UniCarona
Que	é uma plataforma de caronas solidárias e colaborativas exclusiva para a comunidade acadêmica.
Diferente de	aplicativos comerciais de transporte que visam lucro ou grupos de mensagens desorganizados.
Nosso produto	foca no auxílio mútuo e na segurança institucional, permitindo que alunos se ajudem a chegar à universidade sem visar ganhos financeiros, apenas a divisão de custos ou a cortesia.

4. Descrição dos *Stakeholders* e dos Usuários

4.1 Principais *Stakeholders* e Usuários

Identificação	Responsabilidades	*Stakeholders*
Passageiros	Solicitar caronas,realizar cadastro na plataforma, informar rota desejada e avaliar motoristas	Usuários
Motoristas	Oferecer caronas, realizar cadastro na plataforma, registrar rotas disponíveis e avaliar passageiros	Usuários
Equipe de Desenvolvimento	Desenvolver, manter e evoluir a plataforma, garantindo segurança e usabilidade	Interno
Administradores da Plataforma	Gerenciar cadastros de usuários, monitorar rotas e garantir o cumprimento das	Externo

	políticas de uso	

4.2 Necessidades Chave dos *Stakeholders* e dos Usuários

No.	Descrição	Prioridade do Cliente (Crítico, Útil e Importante)	Observações
1	Cadastro rígido de usuários para garantir segurança	Crítico	Pode incluir verificação de identidade(CPF e Facial)
2	Unificação e visualização de rotas disponíveis	Crítico	Filtros por acesso, anel viário, etc.
3	Registro e histórico de todas as viagens realizadas	Importante	Auxilia na segurança e rastreabilidade
4	Matching entre passageiros e motoristas com rotas compatíveis	Crítico	Base do funcionamento do app
5	Sistema de avaliação entre usuários	Útil	Aumenta a confiança na plataforma

5. Visão Geral do Produto

5.1 Perspectiva do Produto

O UniCarona é um sistema independente, desenvolvido para dispositivos móveis, que atua como um facilitador de logística social. Ele não possui integração direta com sistemas de transporte público, mas serve como uma camada de apoio à mobilidade acadêmica.

Necessidades	Funcionalidades Correspondentes
1. Segurança e Verificação de Vínculo	1.1 – Módulo de Cadastro com validação de CPF, e-mail institucional e reconhecimento facial (Facial ID).
2. Encontro de trajetos compatíveis	2.1 Algoritmo de *Matching* que sugere caronas baseadas na proximidade de origem/destino e horários de aula.
3. Confiança entre membros	3.1 Sistema de Gamificação e Avaliação (estrelas e comentários) após cada viagem.

4. Localização em tempo real	4.1 Mapa interativo com rastreamento da carona em curso para monitoramento de segurança.
5. Comunicação interna	5.1 Chat integrado para ajuste de detalhes (ponto de encontro específico) sem expor o número de telefone.
6. Gestão de Rotas	6.1 Cadastro de rotas frequentes para motoristas e painel de solicitações para passageiros.

5.2 Estimativa de Custo Inicial

– Infraestrutura Cloud: Estimativa mensal para hospedagem de banco de dados e servidor de aplicação (AWS ou Azure).

– Serviços de Terceiros: Custos por requisição de API de Mapas (Google Maps/Mapbox) e serviço de autenticação biométrica.

– Mão de Obra: Esforço mensurado em horas para a equipe de desenvolvimento (Frontend, Backend e QA).

– Manutenção: Reserva de 15% do custo de desenvolvimento para correções e atualizações de segurança.

5.3 Suposições e dependências

– Conectividade: O usuário deve possuir um dispositivo móvel com acesso à internet (3G/4G/5G) para o funcionamento do rastreamento.
– Adesão da Comunidade: O sucesso do sistema depende de uma massa crítica de alunos motoristas dispostos a oferecer as vagas ociosas.
– Precisão de GPS: A acurácia do matching depende do sinal de GPS dos dispositivos dos usuários.

5.4 Premissas e Dependências

O sistema deverá utilizar os seguintes padrões de infraestrutura e ferramentas para o seu pleno desenvolvimento e execução:

Máquina do Desenvolvedor:

- **SDK Flutter:** Versão estável (Stable Channel) para o desenvolvimento da interface multi-plataforma.
- **Linguagem Dart:** Versão 3.x ou superior.
- **IDE:** Visual Studio Code ou Android Studio com plugins oficiais do Flutter e Dart.
- **Hardware:** Mínimo de 8GB RAM (16GB recomendado) e processador com suporte a

virtualização (VT-x ou AMD-V) para emulação de dispositivos Android/iOS.

Máquina do Servidor:

- **Ambiente de Execução:** Node.js versão 18.x (LTS) ou superior.
- **Framework Backend:** NestJS (para garantir escalabilidade e organização em módulos).
- **Linguagem:** TypeScript 5.x para tipagem estática e redução de bugs em tempo de execução.
- **Gerenciador de Pacotes:** NPM ou Yarn.

Banco de Dados:

- **Servidor:** PostgreSQL.
- **ORM:** Prisma ou TypeORM (para a ponte entre o Node.js e o PostgreSQL).
- **Requisito de Localização:** Suporte a dados geoespaciais (Geography data type) para o cálculo das rotas.

6. Precedência e Prioridades

Crítico	Sem a validação de identidade (01) e o matching (04), o produto não cumpre sua promessa de segurança e utilidade básica.
Importante	O rastreamento (05) e o histórico (06) são vitais para a confiança institucional, mas dependem de o sistema de carona já estar operante.
Útil	O Chat (08) e as Notificações (10) melhoram a experiência de uso, mas podem ser substituídos por métodos externos (como o próprio contato direto após o match) nas primeiras semanas de teste.

No.	Funcionalidade	Prioridade do Cliente	Entrega
01	**Módulo de Cadastro e Validação (CPF + Facial ID)**	**Crítico**	Entrega 1.0 (MVP)
02	**Cadastro de Rotas e Horários (Motorista)**	**Crítico**	Entrega 1.0 (MVP)
03	**Solicitação de Carona e Busca (Passageiro)**	**Crítico**	Entrega 1.0 (MVP)

04	**Algoritmo de Matching (Cruzamento de Dados)**	**Crítico**	Entrega 1.0 (MVP)
05	**Rastreamento de Viagem em Tempo Real**	**Importante**	Entrega 1.1 (Segurança)
06	**Histórico de Viagens Realizadas**	**Importante**	Entrega 1.1 (Segurança)
07	**Sistema de Avaliação Bidirecional (Rating)**	**Importante**	Entrega 2.0 (Social)
08	**Chat Interno para Combinar Pontos de Encontro**	**Útil**	Entrega 2.0 (Social)
09	**Painel de Estatísticas de Sustentabilidade**	**Útil**	Entrega 3.0 (Engajamento)
10	**Sistema de Notificações Push (Avisos de Saída)**	**Útil**	Entrega 3.0 (Engajamento)

7. Requisitos Não-Funcionais do Produto

Esta seção define os atributos de qualidade, restrições de ambiente e diretrizes de segurança e usabilidade que o aplicativo UniCarona deve atender para garantir uma operação eficiente e segura.

- **Requisitos de Sistema**
  - O aplicativo deve processar o algoritmo de cruzamento de dados (matching) de forma ágil para conectar rapidamente motoristas e passageiros com trajetos compatíveis.
  - O sistema deve suportar requisições simultâneas de localização em tempo real para alimentar o mapa interativo de rastreamento das caronas em curso.
- **Requisitos de Suportabilidade / Ambiente**
  - O aplicativo móvel deve ser desenvolvido utilizando o SDK Flutter em sua versão estável (Stable Channel) com a linguagem Dart (versão 3.x ou superior) para garantir compatibilidade multiplataforma (Android e iOS).
  - O servidor (backend) deve ser executado em ambiente Node.js versão 18.x (LTS) ou superior, utilizando o framework NestJS e a linguagem TypeScript 5.x.
  - A persistência de dados deve ser realizada no banco de dados PostgreSQL Enterprise, com uso obrigatório do tipo de dado espacial (*Geography data type*) para permitir o cálculo preciso das rotas.
- **Requisitos de Usabilidade**
  - A interface de usuário deve ser simples e intuitiva, facilitando a visualização unificada de rotas entre o campus e as cidades satélites.
  - A comunicação entre motorista e passageiro deve ser facilitada por um chat interno integrado ao aplicativo.
- **Requisitos de Confiabilidade**
  - A acurácia do algoritmo de matching e do rastreamento de segurança depende diretamente da precisão da captura do sinal de GPS dos dispositivos móveis dos usuários.
  - O sistema deve apresentar alta disponibilidade, especialmente nos horários de pico que coincidem com o término das aulas e o deslocamento de primeira hora.
- **Requisitos de Segurança**
  - O cadastro na plataforma exige validação rigorosa de identidade, tornando obrigatória a checagem de CPF, e-mail institucional e reconhecimento facial (Facial ID).
  - O sistema deve garantir a privacidade dos usuários durante a negociação de pontos de encontro no chat interno, ocultando o número de telefone de ambas as partes.
  - Todas as viagens devem gerar um registro em um histórico detalhado, visando garantir a rastreabilidade e a segurança institucional da comunidade acadêmica.

- **Requisitos de Documentação**
  - O sistema deve possuir especificações suplementares devidamente documentadas, incluindo o Documento de Regra de Negócio, Documento de Requisitos, Documento Manual do Usuário e o Dicionário de Dados.

8. Restrições Técnicas

Este tópico descreve as limitações técnicas, escolhas arquiteturais obrigatórias e os padrões que a equipe de desenvolvimento deve seguir rigorosamente durante a construção do UniCarona.

8.1 Padrões Aplicáveis

- **Padrões de Interface (UI/UX):** Como o aplicativo será desenvolvido em Flutter, a interface deve adotar as diretrizes visuais do *Material Design* (para dispositivos Android) e *Cupertino* (para dispositivos iOS), garantindo usabilidade familiar e fluida em ambas as plataformas.
- **Arquitetura de Comunicação:** A integração entre o aplicativo móvel e o servidor (backend) deverá ser construída utilizando o padrão de arquitetura API RESTful, padronizando a troca de informações no formato JSON.
- **Padrões de Segurança:** O tráfego de dados sensíveis exigidos pelo sistema, como CPF e validação biométrica, deve ser feito obrigatoriamente sob o protocolo HTTPS (criptografia TLS/SSL). A gestão de sessões ativas dos usuários utilizará o padrão de tokens de segurança (como o JWT - *JSON Web Tokens*).
- **Qualidade e Padrão de Código:** O código desenvolvido deve adotar a tipagem estática rigorosa do TypeScript no backend e as convenções oficiais da linguagem Dart no frontend, facilitando manutenções futuras pela equipe ou por novos mantenedores da universidade.

8.2 Restrições Aplicáveis

- **Stack Tecnológica Engessada:** O projeto está estritamente limitado ao uso das ferramentas homologadas nas premissas: Flutter/Dart para o app , Node.js com NestJS e TypeScript para o backend , e PostgreSQL Enterprise utilizando o *Geography data type* (tipo de dado espacial) para manipulação de rotas.
- **Dependência de APIs Externas:** O sistema possui forte restrição atrelada aos serviços de terceiros. O funcionamento da plataforma depende da disponibilidade e dos limites de requisição de serviços externos de mapas (ex: Google Maps ou Mapbox) e do provedor da API de reconhecimento facial (Facial ID).
- **Restrições de Hardware e Conectividade do Usuário:** O aplicativo não operará de modo offline. É requisito técnico irrevogável que o dispositivo do usuário possua conexão contínua com a internet (3G/4G/5G) e sensor de GPS ativo e preciso para o correto funcionamento do algoritmo de *matching* e do rastreamento de viagens em tempo real.
- **Restrição de Módulos Financeiros:** Em conformidade com o escopo , o sistema não pode possuir integrações com *gateways* de pagamento eletrônico (como processamento de cartões de crédito ou PIX nativo), visto que o aplicativo visa a cortesia e a divisão de custos operada fora da plataforma, caracterizando a carona estritamente solidária.

9. Manual de Arquitetura

9.1 Visão de Implementação

O sistema UniCarona é estruturado seguindo uma arquitetura **Server-Client em camadas**, onde o aplicativo móvel (cliente) se comunica com o servidor (backend) exclusivamente por meio de uma API RESTful, trafegando dados no formato JSON sob protocolo HTTPS.

A implementação está dividida em três camadas principais:

Camada de Apresentação (Frontend): Desenvolvida com Flutter/Dart, responsável por toda a interface gráfica do usuário, compatível com Android e iOS. Segue as diretrizes do Material Design e Cupertino conforme a plataforma. É nessa camada que ocorrem as interações do usuário: cadastro, visualização de rotas, solicitação de carona, chat interno e mapa de rastreamento.

Camada de Negócio (Backend): Desenvolvida com Node.js 18.x (LTS), framework NestJS e linguagem TypeScript 5.x. Responsável por toda a lógica de negócio da plataforma, incluindo o algoritmo de matching, validação de identidade, geração e validação de tokens JWT, comunicação com APIs externas (Google Maps/Mapbox e Facial ID) e regras de segurança.

Camada de Dados: Banco de dados PostgreSQL com suporte ao tipo de dado geoespacial (Geography data type), acessado via ORM (Prisma ou TypeORM). Responsável pela persistência de todos os dados da plataforma: usuários, rotas, viagens, avaliações e histórico.

9.2 Mecanismo Arquiteturais

Mecanismo de Análise	*Mecanismo de Design*	*Mecanismos de Implementação*
Persistência	*Banco de Dados Relacional*	*PostgreSQL*
Comunicação Cliente-Servidor	API RESTful	NestJS + JSON over HTTPS
Segurança de Autenticação	Token de Sessão	JWT (JSON Web Token)
Validação de Identidade	Biometria Facial + CPF	Facial ID API (serviço externo)
Geolocalização e Rotas	Dados Geoespaciais	Google Maps / Mapbox + Geography data type (PostgreSQL)
Interface do Usuário	Design Multiplataforma	Flutter/Dart (Material Design + Cupertino)

Lógica de Negócio	Módulos Orientados a Domínio	NestJS + TypeScript
Comunicação em Tempo Real	WebSocket / Push Notification	Firebase